

Code Variation as a Defense

University of Texas at Dallas
Erick Bauman - exb131030@utdallas.edu
Huibo Wang - hwx142830@utdallas.edu
Travis Wright - tnw130030@utdallas.edu

May 5, 2018

Abstract

This research seeks to determine whether obfuscating the structure of binaries and systems through compiler and kernel modification has any benefit to security

Index terms— Code Variation, Obfuscation, Compiler Alteration, Kernel Alteration

1 Project Summary

1.1 Project Description

The goal of this project is to determine if security can be increased by obfuscating the layout of a binary file or the system itself. By changing the compiler settings, the compiler code, or the kernel code, we will observe changes in binary files and attempt to determine if these changes are sufficient to increase security.

1.2 Team Responsibilities

- Set up a Linux environment, using Ubuntu, with Tiny C Compiler
- Make Modification to Tiny C Compiler
- Build a web server application such as Nginx using the modified compiler and test its functionality and stability
- Analyze the binary for degree of difference as a metric

1.3 Total Budget

An estimated 10 hours a week will be devoted to the project

1.4 Deliverables

- Proposal and literature review
- Progress report and presentation
- Final report and presentation
- A modified version of Tiny C Compiler along with documentation

2 Executive Summary

Title: Code Variation as a Defense

Affiliation: University of Texas at Dallas

Authors: Erick Bauman - exb131030@utdallas.edu
Huibo Wang - hwx142830@utdallas.edu
Travis Wright - tnw130030@utdallas.edu

Index terms— Code Variation, Obfuscation, Compiler Alteration, Kernel Alteration

Summary: The goal of our project is to test the effectiveness of code variation as a defense. By changing the code of a compiler or kernel, we will attempt to change the system in order to thwart or complicate current attacks as well as maintain correct behavior and stability. By studying the amount of variation that we create, its stability and correctness, and its resiliency to modern attacks, we can measure if this research was effective. We will be starting small, working with the Tiny C Compiler and a single web server application, like Nginx, to apply and study the changes.

Contents

1	Project Summary	1
1.1	Project Description	1
1.2	Team Responsibilities	1
1.3	Total Budget	1
1.4	Deliverables	1
2	Executive Summary	2
3	Introduction	4
3.1	Problem Statement	4
3.2	Purpose Statement	4
3.3	Motivation	4
3.4	Impact	4
4	Literature Review	4
4.1	<i>How to Make ASLR Win the Clone Wars: Runtime Re-Randomization</i> [3]	4
4.2	<i>CodeArmor: Virtualizing the code space to counter disclosure attacks</i> [2]	4
4.3	<i>Smashing the Stack for Fun and Profit</i> [4]	5
4.4	<i>The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)</i> [5]	5
4.5	<i>Hacking Blind</i> [1]	5
5	Methods and Procedures	5
5.1	Method Characterization	5
5.2	Procedures and Plan Overview	5
5.3	Schedule	6
5.4	Deliverables	6
5.5	Limitations and Delimitations	6
6	Results to Date	6
6.1	Findings Overview	6
6.2	Detailed Findings	6
7	Issues	7
8	Conclusions and Recommendations	7
9	Future Work	7
10	Biographical Sketches of the Investigators	8
10.1	Erick Bauman	8
10.2	Huibo Wang	8
10.3	Travis Wright	8

3 Introduction

3.1 Problem Statement

As seen in Address Space Layout Randomization, randomization can increase the security of executables. Our objective is to determine if there is any way to obfuscate binaries, beyond similar work that has already been done, to increase security.

3.2 Purpose Statement

This research seeks to determine if randomization of binaries can increase security. We seek to do this by modifying the compiler Tiny C Compiler. We will test these changes by compiling a web server application and verifying that it runs correctly in the modified state created by our changed compiler.

3.3 Motivation

The layout of a system is in most cases well defined. A little research can teach an individual what each register is generally used for, the calling convention of a system, the system call numbers, and the even the rough layout of memory. This knowledge is leveraged by attackers to pivot within a system that has been breached. We set out to study the usefulness of obfuscating these known facts. By changing these common conventions, we seek to secure the system as a whole. If changes to the compiler or kernel can alter a system enough while preserving stability and functionality, then many modern attacks that rely on some assumption regarding the base system will fail.

3.4 Impact

If we successfully demonstrate binary transformations that increase the difficulty of an attacker performing successful attacks, then we have introduced a new method of defense against many common attacks that exist today. By altering the system layout, any attack that relies on assumptions of the underlying memory could be invalidated. This would increase the security of systems, forcing attackers to discover the memory layout of the system, thus taking more time and increasing the chance of discovery before a successful attack.

4 Literature Review

4.1 *How to Make ASLR Win the Clone Wars: Runtime Re-Randomization*[3]

RuntimeASLR is a proposed solution to clone-forking attacks, which leverage the lack of re-randomization of memory after a fork within a process. The application performs taint analysis to find all module pointers within the program. Once the fork is executed and the pointers are found, the modules are shuffled in memory and the recorded pointers are remapped to point to the new address of the originally pointed to module. Evaluation of the application applied to Nginx show that it can detect all pointers are remap them in a way to preserve functionality and prevent clone-fork attacks with the cost of some extra overhead.

4.2 *CodeArmor: Virtualizing the code space to counter disclosure attacks* [2]

CodeArmor is a proposed tool that seeks to defeat code disclosure attacks such as Return-Oriented Programming. It virtualizes the code space, decoupling the pointer values from any concrete locations. In addition, honey gadgets are placed throughout the program to trick attackers and bait them into using invalid gadgets that lead them out of usable memory. CodeArmor also does not need access to hypervisors or source code, simply running as an application that attaches to binaries. It is proven to raise resiliency to code disclosure attacks with a 14.5% overhead for common web server applications.

4.3 *Smashing the Stack for Fun and Profit* [4]

This explains buffer overflow attacks and how exploits against binary code are performed. A programmer mistake in a program can lead to the potential of a buffer overflow, which can be used to write content outside of its intended location. This paper discusses stack overflows, in which a buffer overflow on the stack allows an attacker to overwrite return addresses with values the attacker chooses, and therefore allows the attacker to redirect control to any destination. At the time of this attack, the stack was both writable and executable, meaning that the attacker could write arbitrary bytes onto the stack and overwrite the return address to return directly into those bytes. This *shellcode* can perform arbitrary computation since the attacker can fill the stack with any bytes they choose.

4.4 *The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)* [5]

With the introduction of $W\oplus X$, direct code injection became impossible. However, control flow could still be redirected to existing code in the binary. This introduces an attack against binary applications that is now known as Return-Oriented Programming, which is an extension of earlier attacks that alter a program's control flow to perform attacker-chosen actions. By stringing together known gadgets, or a small series of instructions generally followed by a return instruction, the attacker can overwrite memory and jump to wherever they please. This power allows the attacker to completely hi-jack the control flow of the program. The attacker can then perform almost any action they wish.

4.5 *Hacking Blind* [1]

Hacking Blind introduces a new kind of attack called Blind Return-Oriented Programming. It abuses programs that fork new processes, such as the majority of web server applications, and causes them to leak data one bit at a time based on what input causes the program to crash. Since fork does not re-randomize memory, every fork will have the same memory layout and gadgets can be collected with no knowledge of the underlying program that is being attacked. The collected gadgets can then be leveraged to attack the system much as a traditional Return-Oriented Programming attack would. This attack defeats modern Address Space Layout Randomization and Data Execution Prevention.

5 Methods and Procedures

5.1 Method Characterization

Our research was both constructive and empirical in nature. The problem itself was fairly well defined. We attempted to determine if changing a compiler was a plausible method to increase security. We also implemented these theories to test their validity and success.

5.2 Procedures and Plan Overview

We wanted to take an incremental approach, slowly adding to the complexity of our research. Initially, we experimented with the flag available in Tiny C Compiler to see if any randomization could be created.

We then progressed into compiler modification. The group took several weeks to try and understand Tiny C Compiler before making any changes. Once we started to understand the compiler, we targeted the areas we understood for changes. These changes were then tested by compiling a web server and testing the modified binary.

Time constraints prevented us from achieving all of our initial goals. We have documented what we have done so that future groups can learn from us and move forward with any future work.

5.3 Schedule

Feb 9	Submit Proposal
Feb 16	Set Up Environment & Record Initial Benchmark
Feb 23	Test Changes to Compiler Flags
Mar 2	Test Changes to Compiler Flags
Mar 9	Compiler Modification
Mar 16	Compiler Modification
Mar 23	Progress Report & Presentation
Mar 30	Compiler Modification
Apr 6	Compiler Modification
Apr 13	Compiler Modification
Apr 20	Compiler Modification
Apr 27	Compiler Modification
May 4	Final Report and Presentation

5.4 Deliverables

- Proposal and literature review
- Progress report and presentation
- Final report and presentation
- A modified version of Tiny C Compiler along with documentation

5.5 Limitations and Delimitations

The primary limitation of our research was time. Our initial goal proved to be more time intensive than we had thought. As our research progressed, our plan changed as shrank in scope. Our changes to Tiny C Compiler are limited and our tests were not extensive enough to draw substantial conclusions on this method and its usefulness in the general case.

Our delimitation was assumption on scope of attacks. From the beginning, we have not attempted to protect against Blind Return-Oriented Programming. This attack is able to learn the control flow of a program with no previous knowledge by crashing it repeatedly. Such an attack, by it's nature, ignores randomization of the binary and would likely require extreme randomization to be able to defeat. Such changes were out of scope for a semester long project.

6 Results to Date

6.1 Findings Overview

We were able to make a change to the Tiny C Compiler to, in theory, increase the security of a program. By adding a randomized amount of padding between functions, we can change relative addresses inside the program and confuse attacks that rely on knowledge of the memory layout.

6.2 Detailed Findings

We added a function to TCC, `get_random_offset(unsigned int min, unsigned int max)`. This function first generates a random number between `min` and `max`. It then ensures that it is divisible by four and increases the number if not. This is to ensure that the offset is word aligned, where the alignment is every four bytes. This number is then returned.

We call this function from the `gen_function(Sym *sym)` function, which is responsible for generating new functions in the text section. As the function section is created, we call `get_random_offset(0, 1024)` to generate a random offset between 0 and 1024. This offset is added to the existing padding of the function. Since the offset is four-byte aligned, functions should not suffer from poor performance due to alignment.

In practice, every function will have between 0 and 1024 bytes of padding preceding it. These values will vary between function and will be different every time the program is compiled with our altered compiler.

We tested our changes with an installation of `thttpd` on Ubuntu which was hosting a one megabyte file. Testing was done with `ApacheBench`, performing a total of 10000 requests with a concurrency level of 10. We first compiled `thttpd` with unmodified TCC and ran `ApacheBench`. We then recompiled `thttpd` with our modified compiler and tested it again with the same settings. In both tests, all requests were successfully served and the total time taken for the tests were within one tenth of a second, with unmodified TCC taking 4.490 seconds and modified TCC taking 4.396 seconds.

Percentage of Requests Served within a Certain Time (ms)		
% of requests served	Unmodified TCC	Modified TCC
50%	4	4
66%	4	4
75%	4	4
80%	5	5
90%	5	5
95%	5	5
98%	6	6
99%	7	6
100%	10	10

From the table above, we can see the percentage of requests that were served in some amount of time in milliseconds. These results show that our modified compiler has little to no effect on performance. The mean time for each request with unmodified TCC was 4.490 milliseconds. The same value for our modified TCC was 4.396 milliseconds. These numbers are within the margin of error, again showing that our modifications had almost no effect on performance.

7 Issues

Our first issue was time. With a lofty initial goal, we ran out of time and had to adapt our research goals as we progressed.

The second goal was the complexity of Tiny C Compiler and its lack of documentation. Despite its small size, the compiler is still complex. There is almost no documentation existing for how TCC functions. Function and variables names were often nondescript and made understanding TCC much more difficult, causing us to spend too much time learning TCC.

8 Conclusions and Recommendations

We feel that there is still much work to be done in this area, and we were only able to scratch the surface. Adding random padding around functions introduces more complexities while trying to learn the memory layout of a program. Now, not only with the absolute addresses change as they do with Address Space Layout Randomization, but the relative addresses within the program will also change. While this randomization is not enough to thwart Blind Return-Oriented Programming, due to it not needing any knowledge of the program or its memory, it does make any attack that relies on that knowledge more difficult.

We can also conclude that Tiny C Compiler is not the best option for this project. While it is small, it is poorly documented and difficult to alter. Alternatives such as LLVM, which is designed for extendability, would likely yield more successful results.

9 Future Work

Implementation of random padding in additional areas could yield interesting results. We were only able to apply and test it around functions. Theoretically, the same technique could be used for internal function padding, padding between different kinds of sections, and even within data sections.

Changes to the calling convention should also be further researched. Changing the purpose of each register, like moving the stack pointer into register `EAX`, would make deciphering disassembled code far more difficult. Being able to make these changes randomly has potential to strengthen a program against any attack that relies on knowledge of memory or register layout.

References

- [1] A. Bittau, A. Belay, A. Mashtizadeh, D. Mazières, and D. Boneh. Hacking blind. In *2014 IEEE Symposium on Security and Privacy*, pages 227–242, May 2014.
- [2] X. Chen, H. Bos, and C. Giuffrida. Codearmor: Virtualizing the code space to counter disclosure attacks. In *Security and Privacy (EuroS&P), 2017 IEEE European Symposium on*, pages 514–529. IEEE, 2017.
- [3] K. Lu, W. Lee, S. Nürnberger, and M. Backes. How to make aslr win the clone wars: Runtime re-randomization. In *NDSS*, 2016.
- [4] A. One. Smashing the stack for fun and profit. *phrack magazine*, 49 (14), nov. 1996.
- [5] H. Shacham. The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86). In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 552–561. ACM, 2007.

10 Biographical Sketches of the Investigators

10.1 Erick Bauman

Erick Bauman holds a Bachelor’s degree in Computer Science and is currently pursuing a PhD in Computer Science. He is a research assistant in the Lab for Systems and Software Security at the University of Texas at Dallas, and is researching new techniques for binary rewriting and its applications. He has completed coursework in systems security, advanced programming languages, computer networks, and operating systems.

10.2 Huibo Wang

Huibo Wang holds a Master’s degree in Information System and is currently pursuing a PhD in Computer Science. She is a research assistant in the Lab for Systems and Software Security at the University of Texas at Dallas, and is researching new techniques for System security, especially SGX related problems. She has completed coursework in systems security, operating system, computer networks, wireless network, machine learning, artificial Intelligence.

10.3 Travis Wright

Travis Wright holds a Bachelor’s degree in Computer Science and is currently pursuing a Master’s in Computer Science specializing in Information Assurance. He is a recipient of the National Science Foundation’s CyberCorps: Scholarship for Service and completed an internship with the Department of Defense as a Security Analyst. He has completed coursework in computer networks, wireless networks, network security, information security, and operating systems. He is an associate of (ISC)² toward CISSP.